

尚硅谷大数据技术之 Flume

（作者：尚硅谷研究院）

版本：V5.0

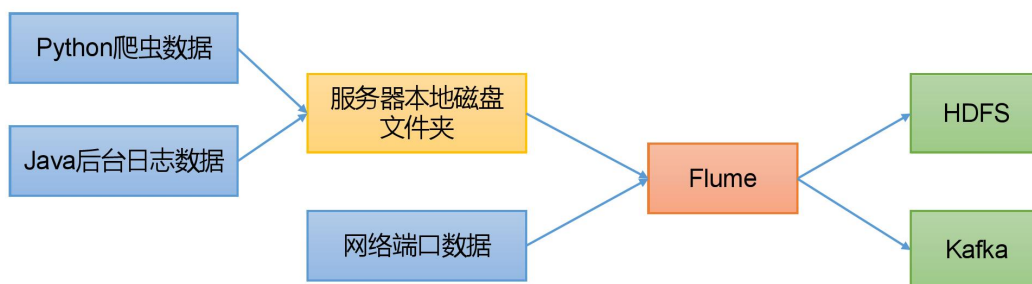
第 1 章 Flume 概述

1.1 Flume 定义

Flume 是 Cloudera 提供的一个高可用的、高可靠的、分布式的海量日志采集、聚合和传输的系统。Flume 基于流式架构，灵活简单。



为什么选用Flume

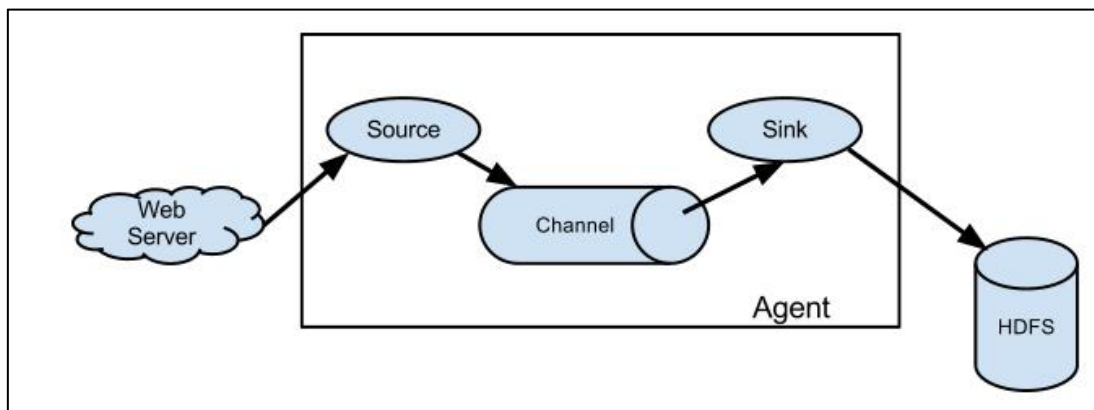


Flume最主要的作用就是，实时读取服务器本地磁盘的数据，将数据写入到HDFS。

让天下没有难学的技术

1.2 Flume 基础架构

Flume 组成架构如下图所示。



1.2.1 Agent

Agent 是一个 JVM 进程，它以事件的形式将数据从源头送至目的地。

Agent 主要有 3 个部分组成，Source、Channel、Sink。

1.2.2 Source

Source 是负责接收数据到 Flume Agent 的组件。Source 组件可以处理各种类型、各种格式的日志数据

1.2.3 Sink

Sink 不断地轮询 Channel 中的事件且批量地移除它们，并将这些事件批量写入到存储或索引系统、或者被发送到另一个 Flume Agent。

1.2.4 Channel

Channel 是位于 Source 和 Sink 之间的缓冲区。因此，Channel 允许 Source 和 Sink 运作在不同的速率上。Channel 是线程安全的，可以同时处理几个 Source 的写入操作和几个 Sink 的读取操作。

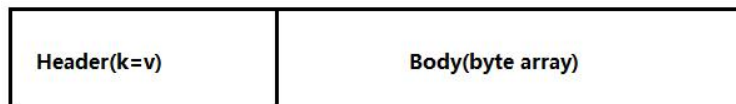
Flume 自带两种 Channel：Memory Channel 和 File Channel。

Memory Channel 是内存中的队列。Memory Channel 在不需要关心数据丢失的情景下适用。如果需要关心数据丢失，那么 Memory Channel 就不应该使用，因为程序死亡、机器宕机或者重启都会导致数据丢失。

File Channel 将所有事件写到磁盘。因此在程序关闭或机器宕机的情况下不会丢失数据。

1.2.5 Event

传输单元，Flume 数据传输的基本单元，以 Event 的形式将数据从源头送至目的地。Event 由 Header 和 Body 两部分组成，Header 用来存放该 event 的一些属性，为 K-V 结构，Body 用来存放该条数据，形式为字节数组。



第 2 章 Flume 入门

2.1 Flume 安装部署

2.1.1 安装地址

- (1) Flume 官网地址: <http://flume.apache.org/>
- (2) 文档查看地址: <http://flume.apache.org/FlumeUserGuide.html>
- (3) 下载地址: <http://archive.apache.org/dist/flume/>

2.1.2 安装部署

- (1) 将 apache-flume-1.10.1-bin.tar.gz 上传到 linux 的/opt/software 目录下。
- (2) 解压 apache-flume-1.10.1-bin.tar.gz 到/opt/module/目录下。

```
[atguigu@hadoop102 software]$ tar -zxvf
/opt/software/apache-flume-1.10.1-bin.tar.gz -C /opt/module/
```

- (3) 修改 apache-flume-1.10.1-bin 的名称为 flume。

```
[atguigu@hadoop102 module]$ mv /opt/module/apache-flume-1.10.1-bin
/opt/module/flume-1.10.1
```

- (4) 修改 conf 下的 log4j2.xml 确定日志打印的位置,在 53 行 21 行后插入。

```
53     <AppenderRef ref="LogFile" />
54     <AppenderRef ref="Console" />

21 <Property
name="LOG_DIR">/opt/module/flume-1.10.1/logs</Property>
```

2.2 Flume 入门案例

2.2.1 监控端口数据官方案例

1) 案例需求:

使用 Flume 监听一个端口,收集该端口数据,并打印到控制台。

2) 实现步骤:

- (1) 安装 netcat 工具。

```
[atguigu@hadoop102 software]$ sudo yum install -y nc
```

- (2) 判断 4444 端口是否被占用。

```
[atguigu@hadoop102 flume]$ sudo netstat -nlp | grep 44444
```

- (3) 在 conf 文件夹下创建 Flume Agent 配置文件 nc-flume-log.conf。

```
[atguigu@hadoop102 conf]$ vim nc-flume-log.conf
```

- (4) 在 nc-flume-log.conf 文件中添加如下内容。

添加内容如下:

```
# Name the components on this agent
a1.sources = r1
a1.sinks = k1
a1.channels = c1
```

```
# Describe/configure the source
a1.sources.r1.type = netcat
a1.sources.r1.bind = localhost
a1.sources.r1.port = 44444

# Describe the sink
a1.sinks.k1.type = logger

# Use a channel which buffers events in memory
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

注：配置文件来源于官方手册 <http://flume.apache.org/FlumeUserGuide.html>

(5) 先开启 flume 监听端口。

第一种写法：

```
[atguigu@hadoop102 flume]$ bin/flume-ng agent --conf conf/ --name a1
--conf-file conf/nc-flume-log.conf -Dflume.root.logger=INFO,console
```

第二种写法：

```
[atguigu@hadoop102 flume]$ bin/flume-ng agent -c conf/ -n a1 -f
conf/nc-flume-log.conf -Dflume.root.logger=INFO,console
```

参数说明：

- --conf/-c: 表示配置文件存储在 conf/目录
- --name/-n: 表示给 agent 起名为 a1
- --conf-file/-f: flume 本次启动读取的配置文件是在 conf 文件夹下的 nc-flume-log.conf 文件。

➤ -Dflume.root.logger=INFO,console : -D 表示 flume 运行时动态修改 flume.root.logger 参数属性值，并将控制台日志打印级别设置为 INFO 级别。日志级别包括:log、info、warn、error。日志参数已经在配置文件中修改了，不再需要重复输入。

(6) 使用 netcat 工具向本机的 44444 端口发送内容。

```
[atguigu@hadoop102 ~]$ nc localhost 44444
hello
atguigu
10
```

(7) 在 Flume 监听页面观察接收数据情况。

```
2021-07-15 13:51:00,236
(SinkRunner-PollingRunner-DefaultSinkProcessor) [INFO -
org.apache.flume.sink.LoggerSink.process(LoggerSink.java:95)]
Event: { headers:{} body: 31 30
10 }
```

(8) event 打印的源码介绍。

LoggerSink 的 process 方法：

```
if (event != null) {
    if (logger.isInfoEnabled()) {
        logger.info("Event: " + EventHelper.dumpEvent(event,
maxBytesToLog));
    }
}
```

dumpEvent 方法返回值: **buffer** 是固定长度的字符串, 前端是 16 进制表示的字符的阿斯卡码值。

```
return "{ headers:" + event.getHeaders() + " body:" + buffer + " }";
```

2.2.2 实时监控目录下的多个追加文件

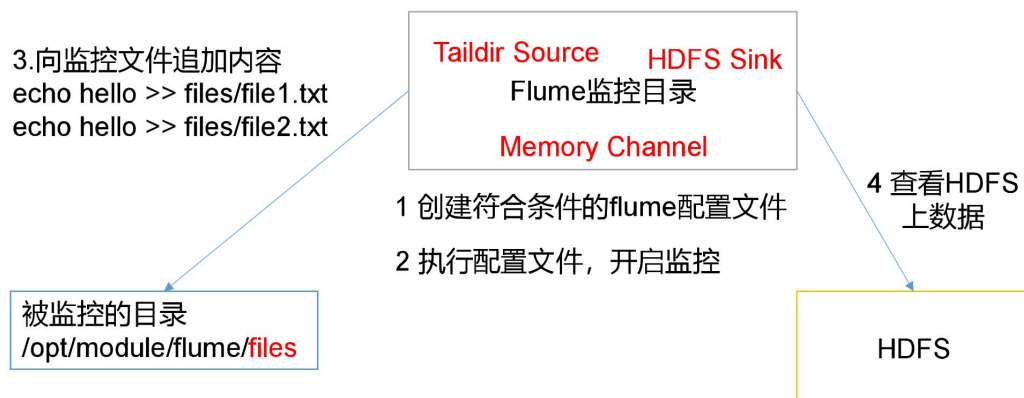
Taildir Source 适合用于监听 **多个实时追加的文件**, 并且能够实现 **断点续传**。

1) 案例需求:使用 Flume 监听整个目录的实时追加文件, 并上传至 HDFS

2) 需求分析



实时读取目录文件到HDFS案例



让天下没有难学的技术

3) 实现步骤

(1) 在 conf 目录下创建配置文件 taildir-flume-hdfs.conf。

① 创建一个文件。

```
[atguigu@hadoop102 conf]$ vim taildir-flume-hdfs.conf
```

② 添加如下内容。

```
a1.sources = r1
a1.sinks = k1
a1.channels = c1

# Describe/configure the source
```

```
a1.sources.r1.type = TAILDIR
a1.sources.r1.filegroups = f1 f2
# 必须精确到文件, 可以写匹配表达式匹配多个文件
a1.sources.r1.filegroups.f1 = /opt/module/flume/files1/*.file.*
a1.sources.r1.filegroups.f2 = /opt/module/flume/files2/*.log.*
# 实现断点续传的文件存放位置 不改有默认位置也能实现断点续传
a1.sources.r1.positionFile = /opt/module/flume/tailldir_position.json

# Describe the sink
a1.sinks.k1.type = hdfs
# 地址值可以填写 hdfs://hadoop102:8020 也可以省略, flume 会自动读取 hadoop 配置文件信息获取地址
a1.sinks.k1.hdfs.path = hdfs://hadoop102:8020/flume/%Y%m%d/%H
# 上传文件的前缀
a1.sinks.k1.hdfs.filePrefix = log-

# 是否使用本地时间戳
a1.sinks.k1.hdfs.useLocalTimeStamp = true

# 设置文件类型 分为二进制文件 SequenceFile 和文本文件 DataStream(不能压缩) 和 CompressedStream(可以压缩)
a1.sinks.k1.hdfs.fileType = DataStream

# 多久生成一个新的文件
a1.sinks.k1.hdfs.rollInterval = 30
# 设置每个文件的滚动大小大概是 128M
a1.sinks.k1.hdfs.rollSize = 134217700
# 文件的滚动与 Event 数量无关
a1.sinks.k1.hdfs.rollCount = 0

# Use a channel which buffers events in memory
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1
a1.sinks.k1.channel = c1
```

(2) 向 files 文件夹中追加内容。

在/opt/module/flume 目录下创建 files1 文件夹。

```
[atguigu@hadoop102 flume]$ mkdir files1
[atguigu@hadoop102 flume]$ mkdir files2
```

(3) 启动监控文件夹命令。

```
[atguigu@hadoop102 flume]$ bin/flume-ng agent --conf conf/ --name a1
--conf-file conf/tailldir-flume-hdfs.conf
```

向 upload 文件夹中添加文件。

```
[atguigu@hadoop102 files1]$ echo hello >> file1.txt
[atguigu@hadoop102 files1]$ echo atguigu >> file2.txt
```

(4) 查看 HDFS 上的数据

Taildir 说明:

Taildir Source 维护了一个 JSON 格式的 position File，其会定期的往 position File 中更新每个文件读取到的最新的位置，因此能够实现断点续传。Position File 的格式如下：

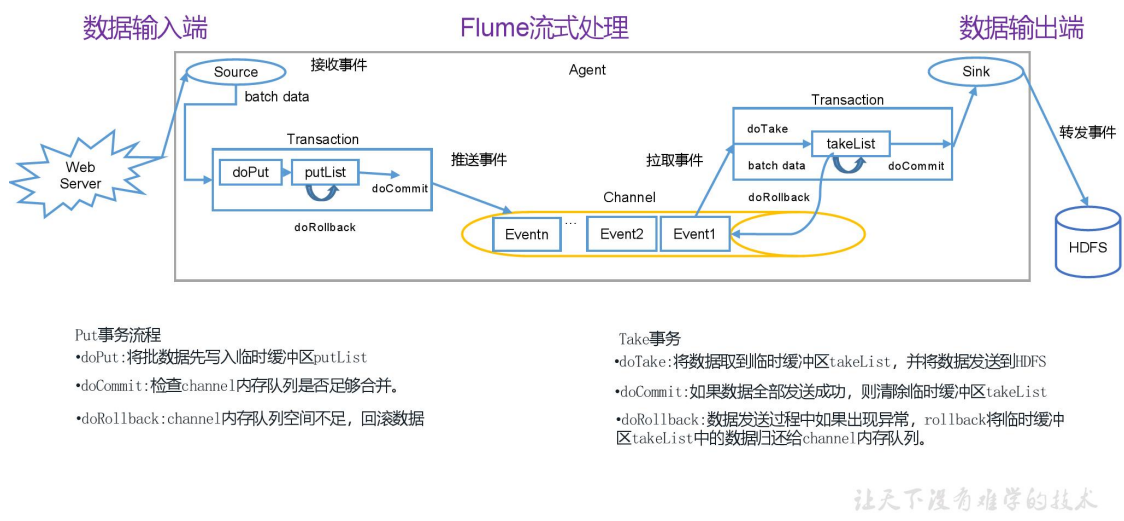
```
{ "inode": 2496272, "pos": 12, "file": "/opt/module/flume/files1/file1.txt" }
{ "inode": 2496275, "pos": 12, "file": "/opt/module/flume/files1/file2.txt" }
```

注：Linux 中储存文件元数据的区域就叫做 inode，每个 inode 都有一个号码，操作系统用 inode 号码来识别不同的文件，Unix/Linux 系统内部不使用文件名，而使用 inode 号码来识别文件。TailDir source 使用 inode 和文件的全路径一起识别同一个文件，所以修改文件名之后如果表达式也能够匹配上，会再重新读取一份文件的数据。

第 3 章 Flume 进阶

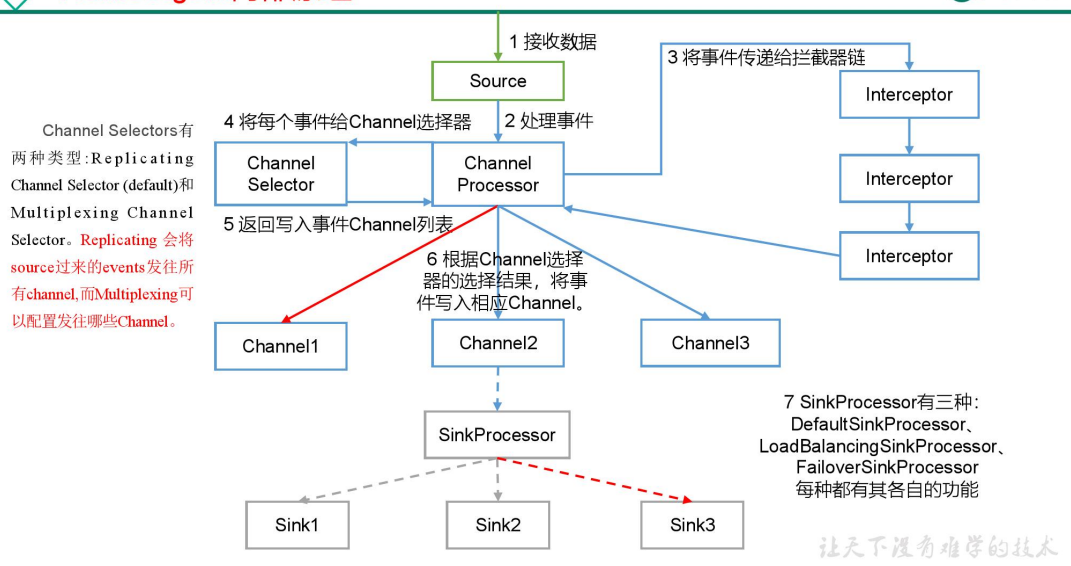
3.1 Flume 事务

Flume 事务



3.2 Flume Agent 内部原理

Flume Agent内部原理



组件介绍:

1) ChannelSelector

ChannelSelector 的作用就是选出 Event 将要被发往哪个 Channel。其共有两种类型，分别是 **Replicating**（复制）和 **Multiplexing**（多路复用）。

ReplicatingSelector 会将同一个 Event 发往所有的 Channel, Multiplexing 会根据相应的原则，将不同的 Event 发往不同的 Channel。

2) SinkProcessor

SinkProcessor 共有三种类型，分别是 **DefaultSinkProcessor**(默认 1 对 1)、**LoadBalancingSinkProcessor**(负载均衡)和 **FailoverSinkProcessor**(故障转移)

DefaultSinkProcessor 对应的是单个的 Sink，LoadBalancingSinkProcessor 和 FailoverSinkProcessor 对应的是 Sink Group，LoadBalancingSinkProcessor 可以实现负载均衡的功能，FailoverSinkProcessor 可以错误恢复的功能。

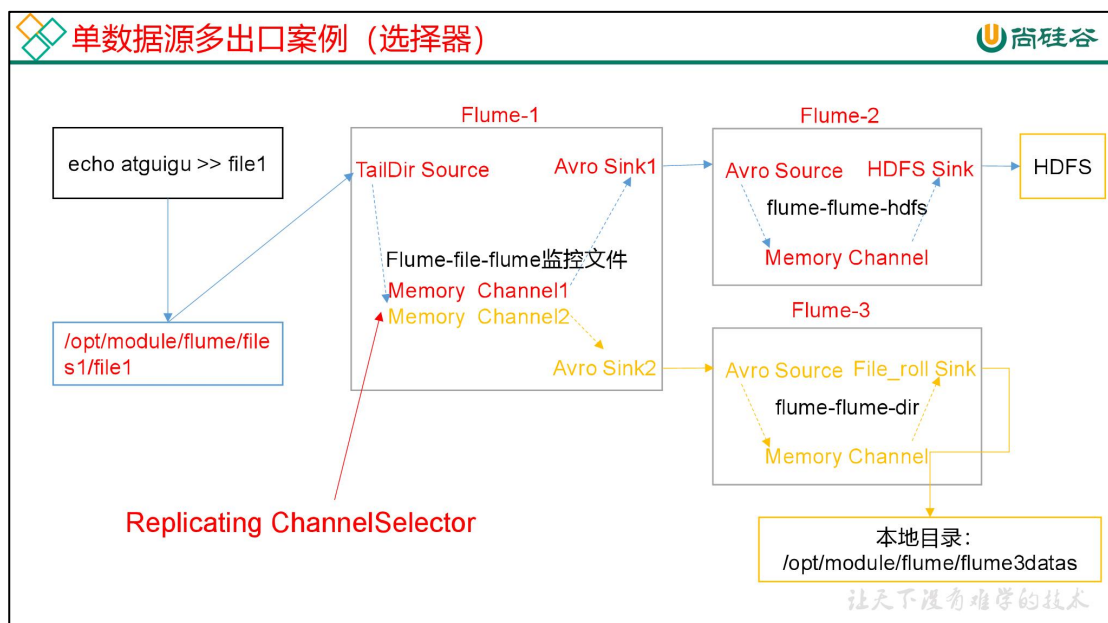
3.3 Flume 企业开发案例

3.3.1 复制案例

1) 案例需求

使用 Flume-1 监控文件变动，Flume-1 将变动内容传递给 Flume-2，Flume-2 负责存储到 HDFS。同时 Flume-1 将变动内容传递给 Flume-3，Flume-3 负责输出到 Local FileSystem。

2) 需求分析



3) 实现步骤

(1) 准备工作。

在/opt/module/flume/conf 目录下创建 group1 文件夹。

```
[atguigu@hadoop102 conf]$ mkdir group1/
```

在/opt/module/flume/目录下创建 flume3datas 文件夹。

```
[atguigu@hadoop102 flume]$ mkdir flume3 datas
```

(2) 创建 flume1.conf。

配置 1 个接收日志文件的 source 和两个 channel、两个 sink, 分别输送给 flume2 和 flume3。

① 编辑配置文件。

```
[atguigu@hadoop102 group1]$ vim flume1.conf
```

② 添加如下内容。

```
# Name the components on this agent
a1.sources = r1
a1.sinks = k1 k2
a1.channels = c1 c2

# Describe/configure the source
a1.sources.r1.type = TAILDIR
a1.sources.r1.filegroups = f1 f2
a1.sources.r1.filegroups.f1 = /opt/module/flume/files1/*.file.*
a1.sources.r1.filegroups.f2 = /opt/module/flume/files2/*.log.*
a1.sources.r1.positionFile = /opt/module/flume/tailedir_position.json

# 将数据流复制给所有 channel 默认参数可以不写
a1.sources.r1.selector.type = replicating

# Describe the sink
# sink 端的 avro 是一个数据发送者
```

```
a1.sinks.k1.type = avro
a1.sinks.k1.hostname = hadoop102
a1.sinks.k1.port = 4141

a1.sinks.k2.type = avro
a1.sinks.k2.hostname = hadoop102
a1.sinks.k2.port = 4142

# Describe the channel
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

a1.channels.c2.type = memory
a1.channels.c2.capacity = 1000
a1.channels.c2.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1 c2
a1.sinks.k1.channel = c1
a1.sinks.k2.channel = c2
```

(3) 创建 flume2.conf。

配置上级 Flume 输出的 Source，输出是到 HDFS 的 Sink。

①编辑配置文件。

```
[atguigu@hadoop102 group1]$ vim flume2.conf
```

②添加如下内容。

```
# Name the components on this agent
a1.sources = r1
a1.sinks = k1
a1.channels = c1

# Describe/configure the source

a1.sources.r1.type = avro
a1.sources.r1.bind = hadoop102
a1.sources.r1.port = 4141

# Describe the sink
a1.sinks.k1.type = hdfs
a1.sinks.k1.hdfs.path = hdfs://hadoop102:8020/flume1/%Y%m%d/%H
# 文件的前缀
a1.sinks.k1.hdfs.filePrefix = log-

#多久生成一个新的文件
a1.sinks.k1.hdfs.rollInterval = 30
#设置每个文件的滚动大小大概是 128M
a1.sinks.k1.hdfs.rollSize = 134217700
#文件的滚动与 Event 数量无关
a1.sinks.k1.hdfs.rollCount = 0
# 使用本地的时间戳
a1.sinks.k1.hdfs.useLocalTimeStamp = true
```

#设置文件类型 分为二进制文件 SequenceFile 和文本文件 DataStream(不能压缩) 和 CompressedStream(可以压缩)

```
a1.sinks.k1.hdfs.fileType = DataStream
```

Use a channel which buffers events in memory

```
a1.channels.c1.type = memory
```

```
a1.channels.c1.capacity = 1000
```

```
a1.channels.c1.transactionCapacity = 100
```

Bind the source and sink to the channel

```
a1.sources.r1.channels = c1
```

```
a1.sinks.k1.channel = c1
```

(4) 创建 flume3.conf。

配置上级 Flume 输出的 Source，输出是到本地目录的 Sink。

① 编辑配置文件。

```
[atguigu@hadoop102 group1]$ vim flume3.conf
```

② 添加如下内容。

Name the components on this agent

```
a1.sources = r1
```

```
a1.sinks = k1
```

```
a1.channels = c1
```

Describe/configure the source

```
a1.sources.r1.type = avro
```

```
a1.sources.r1.bind = hadoop102
```

```
a1.sources.r1.port = 4142
```

Describe the sink

```
a1.sinks.k1.type = file_roll
```

```
a1.sinks.k1.sink.directory = /opt/module/flume/flume3datas
```

Use a channel which buffers events in memory

```
a1.channels.c1.type = memory
```

```
a1.channels.c1.capacity = 1000
```

```
a1.channels.c1.transactionCapacity = 100
```

Bind the source and sink to the channel

```
a1.sources.r1.channels = c1
```

```
a1.sinks.k1.channel = c1
```

提示：输出的本地目录必须是已经存在的目录，如果该目录不存在，并不会创建新的目录。

(5) 启动 Hadoop。

```
[atguigu@hadoop102 hadoop-3.3.4]$ sbin/start-dfs.sh
```

```
[atguigu@hadoop103 hadoop-3.3.4]$ sbin/start-yarn.sh
```

(6) 执行配置文件。

分别启动对应的 flume 进程：flume1、flume2、flume3。

```
[atguigu@hadoop102 flume]$ bin/flume-ng agent --conf conf/ --name a1
--conf-file conf/group1/flume3.conf

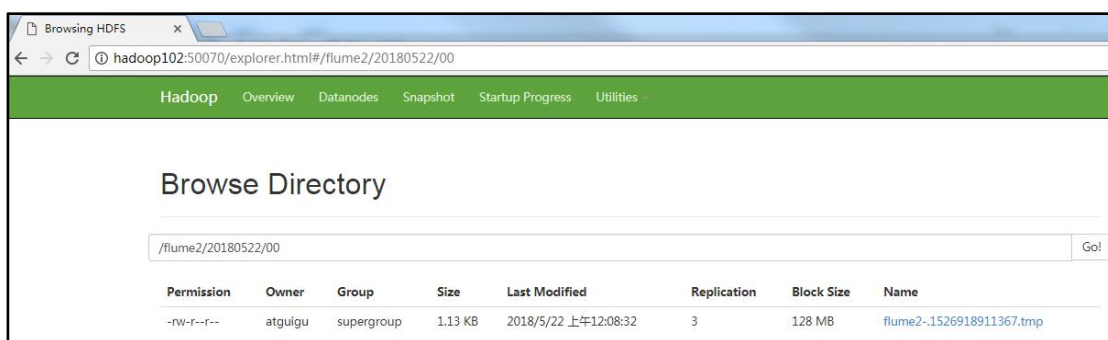
[atguigu@hadoop102 flume]$ bin/flume-ng agent --conf conf/ --name a1
--conf-file conf/group1/flume2.conf

[atguigu@hadoop102 flume]$ bin/flume-ng agent --conf conf/ --name a1
--conf-file conf/group1/flume1.conf
```

(7) 向监控的文件传入数据

```
[atguigu@hadoop102 files1]$ echo hello >> file1.txt
```

(8) 检查 HDFS 上数据



(9) 检查/opt/module/datas/flume3 目录中数据

```
[atguigu@hadoop102 flume3]$ ll
```

总用量 8

```
-rw-rw-r--. 1 atguigu atguigu 5942 5月 22 00:09 1526918887550-3
```

3.3.2 多路复用及拦截器案例

1) 案例需求

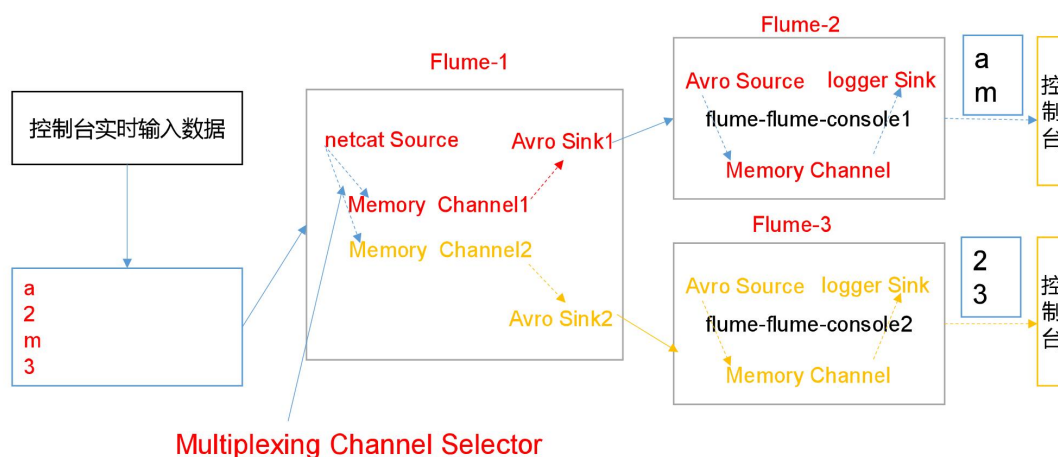
使用 Flume 采集服务器本地日志，需要按照日志类型的不同，将不同种类的日志发往不同的分析系统。

2) 需求分析

在实际的开发中，一台服务器产生的日志类型可能有很多种，不同类型的日志可能需要发送到不同的分析系统。此时会用到 Flume 的 channel selector 中的 Multiplexing 结构，Multiplexing 的原理是，根据 event 中 Header 的某个 key 的值，将不同的 event 发送到不同的 Channel 中，所以我们需要自定义一个 Interceptor，为不同类型的 event 的 Header 中的 key 赋予不同的值。

在该案例中，我们以端口数据模拟日志，以数字（单个）和字母（单个）模拟不同类型的日志，我们需要自定义 interceptor 区分数字和字母，将其分别发往不同的分析系统（Channel）。

Interceptor和Multiplexing ChannelSelector案例



让天下没有难学的技术

3) 实现步骤

(1) 创建一个 maven 项目，并引入以下依赖。

```
<dependencies>
  <dependency>
    <groupId>org.apache.flume</groupId>
    <artifactId>flume-ng-core</artifactId>
    <version>1.10.1</version>
  </dependency>
</dependencies>
```

(2) 定义 CustomInterceptor 类并实现 Interceptor 接口。

```
package com.atguigu.flume;

import org.apache.flume.Context;
import org.apache.flume.Event;
import org.apache.flume.interceptor.Interceptor;

import java.util.List;
import java.util.Map;

/**
 * 1. 实现 interceptor 接口
 * 2. 实现接口的 4 个方法
 * 3. 实现一个静态内部类创建拦截器
 */
public class MyInterceptor implements Interceptor {
```

```
/**
 * 初始化方法
 */
@Override
public void initialize() {

}

/**
 * 处理单条 event
 * @param event
 * @return
 */
@Override
public Event intercept(Event event) {
    // 需要配合 channel 选择器使用 向 headers 当中 put 对应的参数
    // 根据传入的数据 首位是数字还是字母 判断他是不同类型的日志
    byte[] body = event.getBody();
    byte b = body[0];
    Map<String, String> headers = event.getHeaders();

    if (b >= '0' && b <= '9'){
        // b 为数字
        headers.put("type", "number");
    } else if ((b >= 'a' && b <= 'z') || (b >= 'A' && b <= 'Z')) {
        // b 为字母
        headers.put("type", "letter");
    }

    // 可以不需要在写放回 headers
    event.setHeaders(headers);

    return event;
}

/**
 * 处理多条 event
 * @param events
 * @return
 */
@Override
public List<Event> intercept(List<Event> events) {
    for (Event event : events) {
        intercept(event);
    }
    return events;
}

@Override
public void close() {

}

// 静态内部类
public static class MyBuilder implements Builder{
```

```
/**
 * 创建拦截器
 * @return
 */
@Override
public Interceptor build() {
    return new MyInterceptor();
}

@Override
public void configure(Context context) {
}
}
```

(3) 编辑 flume 配置文件

创建新文件 `conf/group2/flume1`，创建新文件 `conf/group2/flume2`, 创建新文件 `conf/group2/flume3`。

为 Flume1 配置 1 个 netcat source，1 个 sink group（2 个 avro sink），并配置相应的 ChannelSelector 和 interceptor。

```
# Name the components on this agent
a1.sources = r1
a1.sinks = k1 k2
a1.channels = c1 c2

# Describe/configure the source
a1.sources.r1.type = netcat
a1.sources.r1.bind = localhost
a1.sources.r1.port = 44444

a1.sources.r1.selector.type = multiplexing
# 使用 headers 中的哪些参数
a1.sources.r1.selector.header = type
a1.sources.r1.selector.mapping.number = c1
a1.sources.r1.selector.mapping.letter = c2

# a1.sources.r1.selector.default = c4

# 拦截器配置
a1.sources.r1.interceptors = i1
a1.sources.r1.interceptors.i1.type = com.atguigu.flume.MyInterceptor$MyBuilder

# Describe the sink

a1.sinks.k1.type = avro
a1.sinks.k1.hostname = hadoop102
a1.sinks.k1.port = 4141

a1.sinks.k2.type = avro
a1.sinks.k2.hostname = hadoop102
```

```
a1.sinks.k2.port = 4142

# Use a channel which buffers events in memory
a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

a1.channels.c2.type = memory
a1.channels.c2.capacity = 1000
a1.channels.c2.transactionCapacity = 100

# Bind the source and sink to the channel
a1.sources.r1.channels = c1 c2
a1.sinks.k1.channel = c1
a1.sinks.k2.channel = c2
为Flume2 配置一个 avro source 和一个 logger sink。
a1.sources = r1
a1.sinks = k1
a1.channels = c1

a1.sources.r1.type = avro
a1.sources.r1.bind = hadoop102
a1.sources.r1.port = 4141

a1.sinks.k1.type = logger

a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

a1.sinks.k1.channel = c1
a1.sources.r1.channels = c1
为Flume3 配置一个 avro source 和一个 logger sink。
a1.sources = r1
a1.sinks = k1
a1.channels = c1

a1.sources.r1.type = avro
a1.sources.r1.bind = hadoop102
a1.sources.r1.port = 4142

a1.sinks.k1.type = logger

a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

a1.sinks.k1.channel = c1
a1.sources.r1.channels = c1
```

(4) 分别启动 flume1, flume2 和 flume3。

(5) 在 hadoop102 使用 netcat 向 localhost:44444 发送字母和数字。

(6) 观察 flume2 和 flume3 打印的日志。

